

ArrayList in Java

The **ArrayList** class in Java is a resizable array, part of the **java.util** package. Unlike arrays, its size can grow dynamically.

Syntax to Create an ArrayList

1. Default Constructor:

```
ArrayList<Type> list = new ArrayList<>();
```

- Creates an empty ArrayList with an initial capacity of 10.

2. With Initial Capacity:

```
ArrayList<Type> list = new ArrayList<>(int initialCapacity);
```

- Creates an ArrayList with a specified initial capacity.

3. From a Collection:

```
ArrayList<Type> list = new ArrayList<>(Collection<? extends Type> c);
```

- Creates an ArrayList containing elements from the specified collection.

Example Code for Creating an ArrayList

1. Default Constructor

```
import java.util.ArrayList;
public class Main
{
    public static void main(String[] args)
    {
        // Creating an ArrayList of Strings
        ArrayList<String> list = new ArrayList<>();

        list.add("Apple");
        list.add("Banana");
        System.out.println(list);           // Output: [Apple, Banana]
    }
}
```

2. With Initial Capacity

```
import java.util.ArrayList;
public class Main
{
    public static void main(String[] args)
    {
        // Creating an ArrayList with an initial capacity of 5
        ArrayList<Integer> numbers = new ArrayList<>(5);

        numbers.add(10);
        numbers.add(20);
        System.out.println(numbers);           // Output: [10, 20]
    }
}
```

3. From a Collection

```
import java.util.ArrayList;
import java.util.Arrays;

public class Main
{
    public static void main(String[] args)
    {
        // Creating an ArrayList from an existing collection
        ArrayList<String> existingList =
            new ArrayList<>(Arrays.asList("Dog", "Cat", "Bird"));

        System.out.println(existingList);      // Output: [Dog, Cat, Bird]
    }
}
```

ArrayList Methods in Java

1. Add Elements

add(E e)

Adds an element to the end of the list.

Example:

```
import java.util.ArrayList;
public class Main
{
    public static void main(String[] args)
    {
        ArrayList<String> list = new ArrayList<>();
        list.add("Apple");
        System.out.println(list);    // Output: [Apple]
    }
}
```

add(int index, E element)

Adds an element at the specified index.

Example:

```
import java.util.ArrayList;
public class Main
{
    public static void main(String[] args)
    {
        ArrayList<String> list = new ArrayList<>();
        list.add("Apple");
        list.add(0, "Banana");
        System.out.println(list);    // Output: [Banana, Apple]
    }
}
```

2. Remove Elements

`remove(Object o)`

Removes the first occurrence of the specified element.

Example:

```
import java.util.ArrayList;
public class Main
{
    public static void main(String[] args)
    {
        ArrayList<String> list = new ArrayList<>();
        list.add("Apple");
        list.add("Banana");
        list.remove("Apple");
        System.out.println(list);    // Output: [Banana]
    }
}
```

`remove(int index)`

Removes the element at the specified index.

Example:

```
import java.util.ArrayList;
public class Main
{
    public static void main(String[] args)
    {
        ArrayList<String> list = new ArrayList<>();
        list.add("Apple");
        list.add("Banana");
        list.remove(1);
        System.out.println(list);    // Output: [Apple]
    }
}
```

3. Access Elements

`get(int index)`

Returns the element at the specified index.

Example:

```
import java.util.ArrayList;
public class Main
{
    public static void main(String[] args)
    {
        ArrayList<String> list = new ArrayList<>();
        list.add("Apple");
        System.out.println(list.get(0));           // Output: Apple
    }
}
```

4. Modify Elements

`set(int index, E element)`

Replaces the element at the specified index with the specified element.

Example:

```
import java.util.ArrayList;
public class Main
{
    public static void main(String[] args)
    {
        ArrayList<String> list = new ArrayList<>();
        list.add("Apple");
        list.set(0, "Banana");
        System.out.println(list);                 // Output: [Banana]
    }
}
```

5. Check Size

size()

Returns the number of elements in the list.

Example:

```
import java.util.ArrayList;
public class Main
{
    public static void main(String[] args)
    {
        ArrayList<String> list = new ArrayList<>();
        list.add("Apple");
        System.out.println(list.size());           // Output: 1
    }
}
```

6. Check Existence

contains(Object o)

Checks if the list contains the specified element.

Example:

```
import java.util.ArrayList;
public class Main
{
    public static void main(String[] args)
    {
        ArrayList<String> list = new ArrayList<>();
        list.add("Apple");
        System.out.println(list.contains("Apple")); // Output: true
    }
}
```

7. Clear List

clear()

Removes all elements from the list.

Example:

```
import java.util.ArrayList;
public class Main
{
    public static void main(String[] args)
    {
        ArrayList<String> list = new ArrayList<>();
        list.add("Apple");
        list.clear();
        System.out.println(list);           // Output: []
    }
}
```

8. Check if Empty

isEmpty()

Checks if the list is empty.

Example:

```
import java.util.ArrayList;
public class Main
{
    public static void main(String[] args)
    {
        ArrayList<String> list = new ArrayList<>();
        System.out.println(list.isEmpty()); // Output: true
    }
}
```

9. Convert to Array

toArray()

Converts the list to an array.

Example:

```
import java.util.ArrayList;
public class Main
{
    public static void main(String[] args)
    {
        ArrayList<String> list = new ArrayList<>();
        list.add("Apple");
        list.add("Banana");
        list.add("Orange");
        Object[] array = list.toArray();
        for (Object obj : array)
        {
            System.out.println(obj);           // Output: Apple
        }
    }
}
```